

Capture the Flag 3: Data-Driven UI

Goal: Separate application data from the UI and handle multiple pieces of data in a usable interface.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- **Explain** an error (not **fix** your error)
- Clarify documentation

You may **NOT** ask AI to generate the solution/code for a flag.

Flag 0 — Create Your Feature Branch

Objective: Create a feature branch before modifying your application.

Create:

`feature/data-ui`

Screenshot:

- Terminal showing your current branch

```
MINGW64:/c:/Users/satom/OneDrive - TAFE NSW/IT314AB_Verano

satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (main)
$ git checkout -b feature/data-ui
Switched to a new branch 'feature/data-ui'

satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (feature/data-ui)
$ git branch
* feature/data-ui
  feature/widgets
  main

satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (feature/data-ui)
$ git pull origin feature/widgets
From https://github.com/KVerano/IT314AB_Verano
 * branch          feature/widgets -> FETCH_HEAD
Already up to date.
```

Flag 1 — Identify the Data

Objective: Identify the appropriate Dart data type for the information in your profile.

Research basic Dart data types and identify at least **four different types** that could be used in your application.

Examples:

String
int
double
bool

Create a table identifying the information and its appropriate data type.

Screenshot:

- Your identified data
- The corresponding data types

Identified Data	Data Types
Name	String
Course	String
Age	Int
Hobby	String
Height	Double
Student Status	Bool

Flag 2 — Create Your Data

Objective: Create appropriately typed variables for your profile.

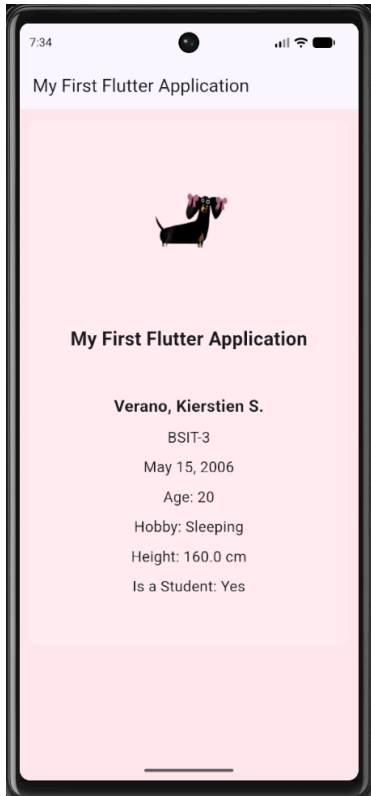
Create variables for at least:

- Name
- Course & Section
- Age
- Hobby
- Height
- Student status

Your data should exist **separately from your UI widgets**.

Screenshot:

- Your variables
- Their data types
- Application still running



```
1  import 'package:flutter/material.dart';
2
3  String appTitle = "My First Flutter Application";
4  String name = "Verano, Kierstien S.";
5  String courseSection = "BSIT-3";
6  String birthdate = "May 15, 2006";
7  int age = 20;
8  String hobby = "Sleeping";
9  double height = 160.0;
10 bool isStudent = true;
11
12   Run | Debug | Profile
13   void main() {
14     runApp(const MyApp());
15   }
```

Flag 3 — Connect Data to UI

Objective: Replace your hardcoded information with your variables.

Your profile should now display information using the variables you created.

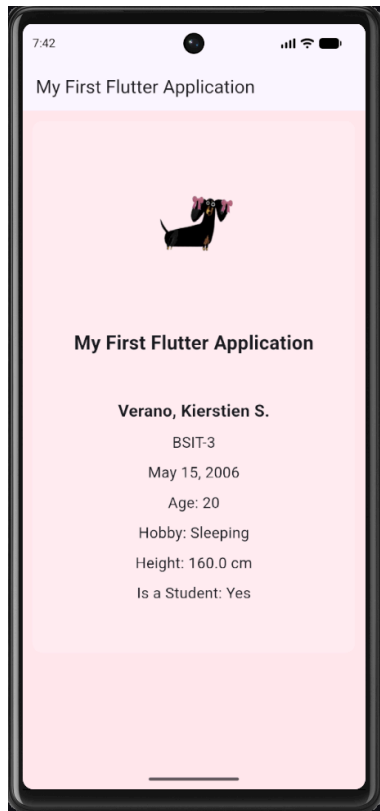
Test:

Change the value of one of your variables.

The UI should automatically display the new value.

Screenshot:

- Variables
- Widgets displaying the variables
- Updated application



```

45   SizedBox(height: 10),
46   Text(
47     appTitle,
48     style: TextStyle(
49       fontSize: 23,
50       fontWeight: FontWeight.bold,
51     ), // TextStyle
52   ), // Text
53
54   SizedBox(height: 50),
55   Text(
56     name,
57     style: TextStyle(
58       fontSize: 20,
59       fontWeight: FontWeight.bold,
60     ), // TextStyle
61   ), // Text
62
63   SizedBox(height: 10),
64   Text(courseSection, style: TextStyle(fontSize: 18)),
65
66   SizedBox(height: 10),
67   Text(birthdate, style: TextStyle(fontSize: 18)),
68
69   SizedBox(height: 10),
70   Text(
71     "Age: ${age.toString()}",
72     style: TextStyle(fontSize: 18),
73   ), // Text
74
75   SizedBox(height: 10),
76   Text("Hobby: $hobby", style: TextStyle(fontSize: 18)),
77

```

```

78   SizedBox(height: 10),
79   Text(
80     'Height: $height cm',
81     style: TextStyle(fontSize: 18),
82   ), // Text
83
84   SizedBox(height: 10),
85   Text(
86     isStudent ? 'Is a Student: Yes' : 'Is a Student: No',
87     style: TextStyle(fontSize: 18),
88   ), // Text

```

🚩 Flag 4 — Make the Image Data-Driven

Objective: Make your profile image part of your data.

Create a variable containing the path to your profile image.

Use the variable when displaying the image.

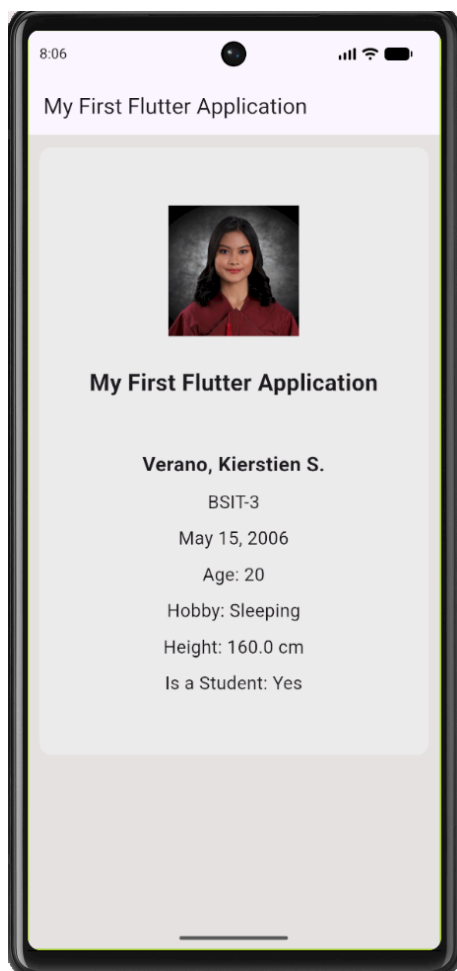
Then change the variable to another registered image.

The image displayed by the application should change.

Screenshot:

- Image variable
- Image widget using the variable
- Updated application

PS: ma'am hehe ako gi change ang color here kay batian ko sa ako pag last huehue



```
3 String profileImage = 'assets/dachshund.png';
4 String appTitle = "My First Flutter Application";
5 String name = "Verano, Kierstien S.";
6 String courseSection = "BSIT-3";
7 String birthdate = "May 15, 2006";
8 int age = 20;
9 String hobby = "Sleeping";
10 double height = 160.0;
11 bool isStudent = true;
```

```
3 String profileImage = 'assets/kierstien.jpeg';
4 String appTitle = "My First Flutter Application";
5 String name = "Verano, Kierstien S.";
6 String courseSection = "BSIT-3";
7 String birthdate = "May 15, 2006";
8 int age = 20;
9 String hobby = "Sleeping";
10 double height = 160.0;
11 bool isStudent = true;
```

```
41 child: Column(
42   mainAxisAlignment: MainAxisAlignment.center,
43   children: [
44     Image.asset(profileImage, width: 130),
45   ],
```



Flag 5 — Five Profiles, One Design

Objective: Create at least **five different profile cards** using the same visual design.

Each profile must contain at least:

- Image
- Name
- Course & Section
- Age
- Hobby

The five profiles must contain different information.

Important

All five profiles should use the **same visual design**.

You are demonstrating that the same UI can represent different data.

Screenshot:

- Your 5 different profile data

```
13 class Profile {
14     String image;
15     String name;
16     String courseSection;
17     int age;
18     String hobby;
19
20     Profile({
21         required this.image,
22         required this.name,
23         required this.courseSection,
24         required this.age,
25         required this.hobby,
26     });
27 }
28
```

```
29 List<Profile> profiles = [
30     Profile(
31         image: 'assets/kierstien.jpeg',
32         name: 'Verano, Kierstien S.',
33         courseSection: 'BSIT-3',
34         age: 20,
35         hobby: 'Sleeping',
36     ),
37
38     Profile(
39         image: 'assets/kyla.jpeg',
40         name: 'Caballero, Kyla Marie S.',
41         courseSection: 'BSIT-3',
42         age: 20,
43         hobby: 'Drawing',
44     ),
45
46     Profile(
47         image: 'assets/zelon.jpeg',
48         name: 'Estimo, Zelon Matthew C.',
49         courseSection: 'BSIT-3',
50         age: 21,
51         hobby: 'Dancing',
52     ),
53
54     Profile(
55         image: 'assets/leila.jpeg',
56         name: 'Bangoy, Leila G.',
57         courseSection: 'BSIT-3',
58         age: 20,
59         hobby: 'Watching Movies',
60     ),
61
62     Profile(
63         image: 'assets/cassy.jpeg',
64         name: 'Oraiz, Cassandra Gayle R.',
65         courseSection: 'BSIT-3',
66         age: 20,
67         hobby: 'Gaming',
68     ),
69 ];
70
```



Flag 6 — THE HIDDEN PROBLEM

You didn't see your UI?

Objective: Make all five profiles accessible to the user.

You just added five profile cards.

Now run your application.

Something is wrong.

Look carefully at your emulator.

Can you see all five profiles?

Maybe you can see the first one...

Maybe part of the second...

But where are the rest?

That's right.

You can't see them.

Your UI is larger than the available screen.

The profiles exist.

The data exists.

The widgets exist.

But the user can't access everything.

Your Mission:

Figure out how Flutter allows users to **scroll through content that is larger than the available screen**.

Research Flutter's **scrolling widgets** and determine which one is appropriate for your current layout.

You must make it possible to scroll through all five profiles.

Requirement:

The user must be able to:

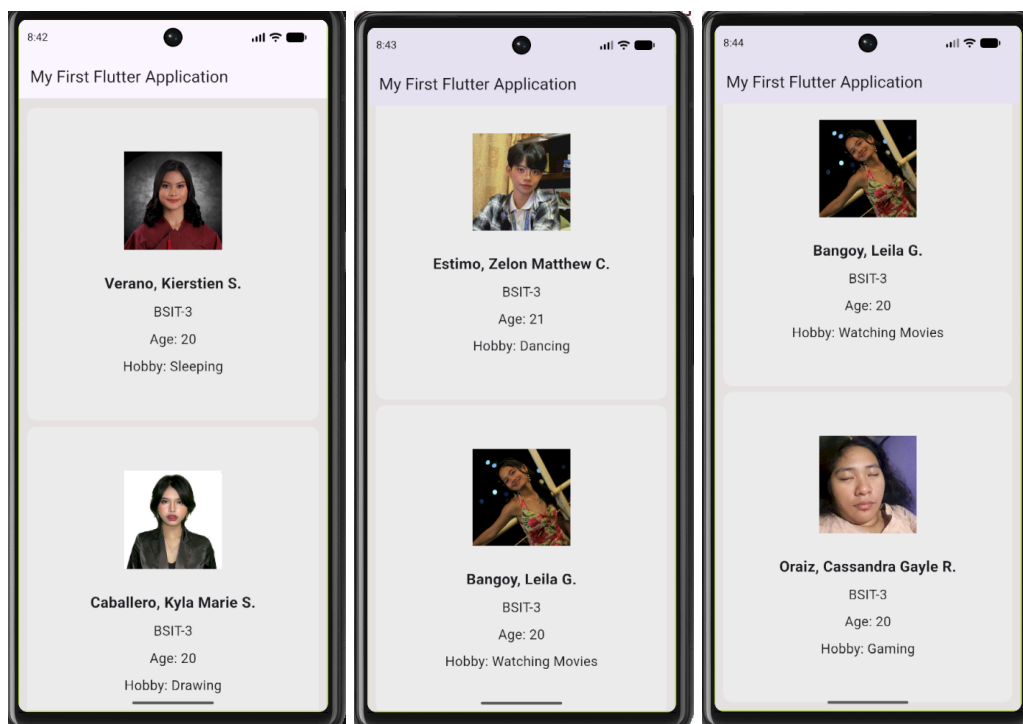
Scroll → see Profile 1 → Profile 2 → Profile 3 → Profile 4 → Profile 5

without changing the size of the screen.

Screenshot:

- The scrolling widget in your code
- Top portion of the application
- The rest of the portion of the application showing the later profiles

```
124 | body: SingleChildScrollView(  
125 |   // provides scrolling capability to a single child container whose content might exceed the available screen space
```



Flag 7 — Survive the Database

Objective: Prepare your UI for incomplete data.

Now imagine your profiles aren't manually created.

Someday, the data will come from a database.

And real data isn't always perfect.

Some information may be:

- Missing
- Empty
- `null`
- Not provided

Create at least **three profiles with missing information**.

For example:

Profile A

Name: Maria

Course: BSIT

Hobby: **missing**

Profile B

Name: Juan

Course: **missing**

Hobby: Gaming

Profile C

Name: **missing**

Course: MMA

Hobby: Drawing

Your application must **not crash** when information is missing.

Instead, provide an appropriate fallback.

Examples:

Hobby: Not provided

Course: Unknown

Challenge

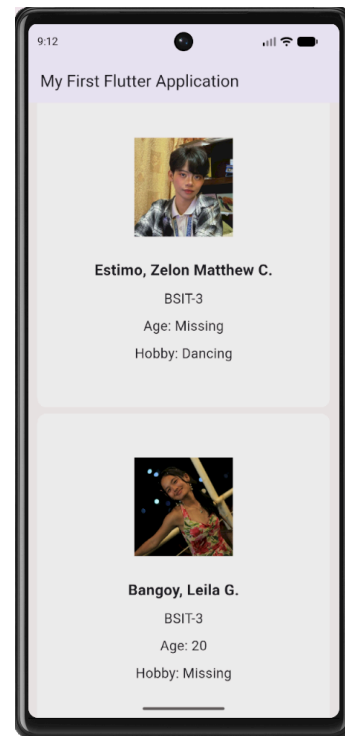
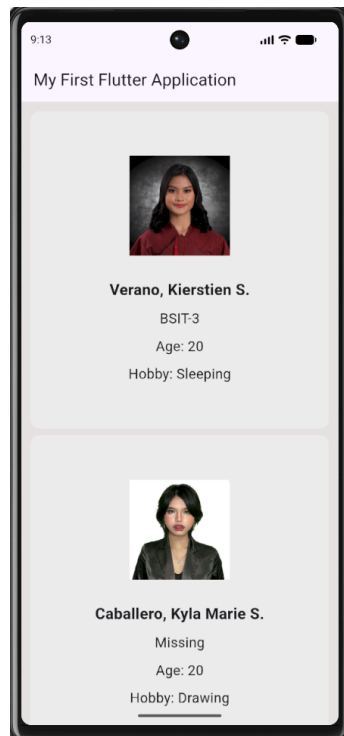
Your placeholders should look intentional and clean.

Screenshot:

- Incomplete profile data
- Application displaying fallback values
- Application still functioning correctly

```
13 class Profile {
14   String? image;
15   String? name;
16   String? courseSection;
17   int? age;
18   String? hobby;
19
20   Profile({this.image, this.name, this.courseSection, this.age, this.hobby});
21 }
22
```

```
32 Profile(
33   image: 'assets/kyla.jpeg',
34   name: 'Caballero, Kyla Marie S.',
35   courseSection: null,
36   age: 20,
37   hobby: 'Drawing',
38 ),
39
40 Profile(
41   image: 'assets/zelon.jpeg',
42   name: 'Estimo, Zelon Matthew C.',
43   courseSection: 'BSIT-3',
44   age: null,
45   hobby: 'Dancing',
46 ),
47
48 Profile(
49   image: 'assets/leila.jpeg',
50   name: 'Bangoy, Leila G.',
51   courseSection: 'BSIT-3',
52   age: 20,
53   hobby: null,
54 ),
55
```



```

83     Image.asset(profile.image ?? 'Missing', width: 130),
84
85     SizedBox(height: 30),
86
87     Text(
88       profile.name ?? 'Missing',
89       style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
90     ), // Text
91
92     SizedBox(height: 10),
93
94     Text(
95       profile.courseSection ?? 'Missing',
96       style: TextStyle(fontSize: 18),
97     ), // Text
98
99     SizedBox(height: 10),
100
101     Text(
102       "Age: ${profile.age ?? 'Missing'}",
103       style: TextStyle(fontSize: 18),
104     ), // Text
105
106     SizedBox(height: 10),
107
108     Text(
109       "Hobby: ${profile.hobby ?? 'Missing'}",
110       style: TextStyle(fontSize: 18),
111     ), // Text

```

Flag 8 — Save Your Progress

Objective: Commit and push your completed Data-Driven UI CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/data-ui`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

```
satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (feature/data-ui)
$ git branch
* feature/data-ui
  feature/widgets
  main
```

```
satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (feature/data-ui)
$ git status
On branch feature/data-ui
Your branch is ahead of 'origin/feature/data-ui' by 2 commits.
(use "git push" to publish your local commits)
```

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   FlutterProjects/my_first_flutter_app/lib/main.dart
        modified:   FlutterProjects/my_first_flutter_app/pubspec.yaml
```

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        FlutterProjects/my_first_flutter_app/assets/cassy.jpeg
        FlutterProjects/my_first_flutter_app/assets/kierstien.jpeg
        FlutterProjects/my_first_flutter_app/assets/kyla.jpeg
        FlutterProjects/my_first_flutter_app/assets/leila.jpeg
        FlutterProjects/my_first_flutter_app/assets/zelon.jpeg
```

no changes added to commit (use "git add" and/or "git commit -a")

```
satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (feature/data-ui)
$ git add .
```

```
satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (feature/data-ui)
$ git commit -m "feat: submit Data-Driven UI"
[feature/data-ui 6a43e44] feat: submit Data-Driven UI
7 files changed, 119 insertions(+), 154 deletions(-)
create mode 100644 FlutterProjects/my_first_flutter_app/assets/cassy.jpeg
create mode 100644 FlutterProjects/my_first_flutter_app/assets/kierstien.jpeg
create mode 100644 FlutterProjects/my_first_flutter_app/assets/kyla.jpeg
create mode 100644 FlutterProjects/my_first_flutter_app/assets/leila.jpeg
create mode 100644 FlutterProjects/my_first_flutter_app/assets/zelon.jpeg
```

```
satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (feature/data-ui)
$ git push
Enumerating objects: 29, done.
Counting objects: 100% (27/27), done.
Delta compression using up to 20 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (15/15), 2.20 MiB | 349.00 KiB/s, done.
Total 15 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 3 local objects.
To https://github.com/KVerano/IT314AB_Verano.git
   50f393c..6a43e44  feature/data-ui -> feature/data-ui
```

